Session 6: Practical Workshop - Finalizing the "Task Tracker" Application

Chapter 1: Building the Skeleton (HTML5) - The Foundation for Automation 1
- Step 1: Boilerplate and Header (Navigation) 2
- Step 2: The Input Section (Form) and Testing Attributes 3
- Step 3: Preparing the List and the Latency Indicator (Spinner) 4

Chapter 2: Visual Presentation (CSS3) - Building the interface step by step 5
- Step 1: Reset and Flexbox on the Navbar 5
- Step 2: The Form and Button States (:disabled vs :hover) 7
- Step 3: Preparing the Task List (Latency & List) 8

Chapter 3: Application Logic (JavaScript) - The Asynchronous "Brain" 10
- Step 1: DOM Selectors and the Rendering Function (Reusability) 10
- Step 2: Loading initial data on startup (GET) 11
- Step 3: Saving a New Task (POST & Latency) 13
- Step 4: Form Validation (the 'input' event) 14

Chapter 4: Network Deletion (DELETE) and the Final QA Checklist 16
- Step 1: Real Deletion (HTTP DELETE Method & Event Delegation) 16
- Step 2: Manual QA Checklist (SUT Sign-off) 18

Chapter 5 (Bonus): "Chaos" Mode - The enemy of automated tests (Flaky Pop-up) 19
- Step 1: Pop-up Styling (Z-Index and Overlays) 20
- Step 2: Random Asynchronous JavaScript (The Injection) 21
- QA Conclusion (SUT Sign-off) 22

Chapter 1: Building the Skeleton (HTML5) - The

Foundation for Automation

🧠 Session 6 is the **Practical Workshop (Capstone Project for Module 2)**. Here we take absolutely everything we've accumulated in Sessions 1 - 5 and assemble it into a single fully functional web application: **the final Task Tracker**. This application will become the "subject" (SUT - Software Under Test) that you will test automatedly starting in Session 10!
We won't copy-paste a huge file. We will build the application structure step by step to understand *why* each line of code matters for Playwright.

🟤 Step 1: Boilerplate and Header (Navigation)

Live Scenario: Any application needs a starting point (the document type declaration) and a header. In the header, we will put the title and a "Network Status" indicator (which will be useful later for latency tests).

🤔 ❓ **How to actually test it:**

- Open VS Code and create a new file named **index.html**.
- Add the code below (Basic structure and Header).
- Save and open the file in the browser (Chrome).
- On the screen, you will only see the raw text "QA Task Tracker" and "API Online". The application has no life yet.

Code to write in index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>QA Task Tracker Pro</title>
    <!-- Preparing the link for the CSS in Chapter 2 -->
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- Application Header -->
```

```
    <header class="navbar">
        <h1 id="logo">QA Task Tracker</h1>
        <div class="user-controls">
            <span id="network-status" class="status-indicator online">API
Online</span>
        </div>
    </header>

    <!-- Main container where we will put the rest of the app -->
    <main class="container">
```

🟤 Step 2: The Input Section (Form) and Testing Attributes

Live Scenario: Now we add the area where the user will enter text. This is the most important step for a QA! Notice the data-testid attributes. We are building the application keeping *testability* in mind right from second zero. Playwright will never have to guess where to click, because we just drew the perfect map for it!

🤓 ❓ **How to actually test it:**

- Continue writing the code below in index.html, right under <main class="container">.
- Save and refresh the browser.
- Notice the "Save" button. Even if it looks ugly right now (without CSS), try clicking on it. You can't! Thanks to the native HTML disabled attribute, we blocked sending empty forms right from the start.

Code to continue in index.html:

```
        <!-- Task addition section -->
        <section class="card input-section">
            <h2>Add a New Scenario</h2>
            <form id="add-task-form">
                <div class="input-group">
                    <!-- Text field with a hook for Playwright -->
                    <input type="text" id="task-name" placeholder="e.g.,
Test login validation..." data-testid="input-task-name">

                    <!-- The button starts disabled to force the user to
```

```
type text -->
                    <button type="submit" id="submit-task-btn" data-
testid="submit-task-btn" disabled>Save</button>
                </div>

                <!-- Hidden error message by default (we will activate it
via JS) -->
                <small id="error-msg" class="error-text" style="display:
none;" data-testid="error-message">Task name cannot be empty!</small>
            </form>
        </section>
```

🟢 Step 3: Preparing the List and the Latency Indicator (Spinner)

Live Scenario: Your robot (Playwright) will interact with the network. The network has delays. We must prepare a visual element (#loading-spinner) that appears while we wait for the server response, and an empty list (<ul>) that will be populated by JavaScript later.

🤔 ❓ How to actually test it:

- Add this final block of code in index.html to close the structure.
- Save and refresh the browser.
- You will see the text "Loading data from server...". This is the initial, raw state of our application. Now that the skeleton is ready, we are prepared to move on to the design!

Final code to add in index.html:

```
        <!-- Task list section -->
        <section class="card list-section">
            <h2>Test Scenarios (Live)</h2>

            <!-- Visual element for Asynchronous Latency -->
            <div id="loading-spinner" class="spinner" data-testid="loading-
spinner">Loading data from server...</div>

            <!-- Empty list: will be populated by JavaScript in Chapter 3 -
->
            <ul id="task-list" class="task-list" data-testid="task-list">
                <!-- The rows (<li>) will be injected here by JavaScript --
```

```
>
            </ul>
        </section>

    </main>

    <!-- Preparing the link for the Logic in Chapter 3 -->
    <script src="app.js"></script>
</body>
</html>
```

Once you have saved the file and opened it in the browser, you will only see simple, unformatted text.

🤔 The application has no life and no design yet.

Chapter 2: Visual Presentation (CSS3) - Building the interface step by step

> 🎯 Now that we have the raw structure (HTML) from Chapter 1, if you look in the browser, the application looks like a document from 20 years ago. For our E2E tests to run in a stable environment, similar to what a real user sees in production, we need to apply styling.
> Here we won't copy-paste a huge file. We will build the design step by step directly in the style.css file, to see live how each block of CSS code solves a technical UI problem.

🟤 Step 1: Reset and Flexbox on the Navbar

Live Scenario: The browser's default margins mess up our alignment. Furthermore, we want the "Logo" and the "Status" in the navigation bar to be pushed to the extreme edges of the screen, so they don't overlap on small resolutions.

🤔 ❓ **How to actually test it:**

- In VS Code, create a **style.css** file in the same folder as **index.html**.
- Add the code below and save.
- Refresh the page in the browser. The top bar will become dark blue, and the text will be perfectly aligned left and right.
- **QA Test (Responsiveness):** Shrink the browser window left and right. You will see that thanks to justify-content: space-between, the elements never overlap, keeping a safe distance for the Playwright robot!

Code to write in style.css:

```
/* 1. Basic Reset */
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    font-family: 'Segoe UI', Tahoma, sans-serif;
}

body {
    background-color: #f0f2f5;
    color: #333;
}

/* 2. Navigation Bar (Flexbox) */
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background-color: #1a252f;
    color: white;
    padding: 15px 30px;
}

.status-indicator {
    font-size: 12px;
```

```
    padding: 5px 10px;
    border-radius: 20px;
    font-weight: bold;
}
.online { background-color: #27ae60; color: white; }
```

🟤 Step 2: The Form and Button States (:disabled vs :hover)

Live Scenario: The input section looks messy. More importantly for us as testers: the "Save" button is disabled in HTML (disabled), but visually it looks exactly like a normal button. We need to clearly change its visual state!

🤓 ❓ How to actually test it:

- Add the code below continuing in style.css and refresh the browser.
- Look at the "Save" button. It is now gray, and the cursor changes to a "forbidden" sign (not-allowed) when you hover over it. This is a clear visual indication that the HTML disabled state is working.
- The error message ("Task name cannot be empty!") received the red color specific to alerts.

Code to add in style.css:

```
/* 3. Main Container and Cards */
.container { max-width: 800px; margin: 40px auto; padding: 0 20px; }

.card {
    background: white;
    padding: 25px;
    border-radius: 10px;
    box-shadow: 0 4px 10px rgba(0,0,0,0.05);
    margin-bottom: 30px;
}
.card h2 { margin-bottom: 20px; font-size: 20px; border-bottom: 2px solid
#ecf0f1; padding-bottom: 10px; }

/* 4. Input Form */
.input-group { display: flex; gap: 15px; }

input[type="text"] {
    flex: 1;
    padding: 12px;
    border: 2px solid #dfe6e9;
```

```
    border-radius: 6px;
    font-size: 16px;
}

button {
    background-color: #3498db;
    color: white;
    border: none;
    padding: 12px 25px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: bold;
}

/* Normal Hover State */
button:hover:not(:disabled) { background-color: #2980b9; }

/* "Disabled" State - crucial for UI tests */
button:disabled {
    background-color: #bdc3c7;
    cursor: not-allowed;
}

.error-text { color: #e74c3c; margin-top: 8px; font-size: 13px; font-
weight: bold; }
```

🟢 Step 3: Preparing the Task List (Latency & List)

Live Scenario: The visual "Latency" element (Spinner) needs to stand out to simulate network waiting. Also, we must prepare the CSS for the future <li> rows that JavaScript will generate, ensuring the "Delete" button is always glued to the right edge.

🤔 ❓ How to actually test it:

- Add the final code in style.css.
- Refresh. The message "Loading data..." now appears as a distinct orange banner.
- *Manual Testing Trick:* To see what a JS-generated task will look like, go to index.html for a second and manually add between the <ul id="task-list"> ... tags the following line: <li class="task-item">Manual Test<button class="btn-delete">Delete</button>

- Refresh. You will see that thanks to justify-content: space-between on the .task-item class, the red delete button sits perfectly aligned to the right! (Don't forget to delete the manual line from HTML after you are convinced it works).

Code to add in style.css:

```
/* 5. Visual Feedback for async actions (Spinner) */
.spinner {
    color: #f39c12;
    text-align: center;
    margin: 20px 0;
    padding: 10px;
    background-color: #fcf3cf;
    border-radius: 6px;
}

/* 6. Task List (Preparation for JS) */
.task-list { list-style: none; }

.task-item {
    display: flex;
    justify-content: space-between; /* Pushes text to the left and button
to the right */
    align-items: center;
    padding: 15px 20px;
    background-color: #f8f9fa;
    border: 1px solid #e9ecef;
    border-radius: 6px;
    margin-bottom: 12px;
}

.task-title { font-size: 15px; font-weight: 500; }

.btn-delete { background-color: #e74c3c; font-size: 13px; padding: 8px
15px; }
.btn-delete:hover { background-color: #c0392b; }
```

Our application now has an Enterprise "face"! The next step is to mount its "brain" in Chapter 3.

Chapter 3: Application Logic (JavaScript) - The Asynchronous "Brain"

> 🎯 We have the structure (HTML) and we have the design (CSS). But our application is like a beautiful car without an engine. Now we will write the app.js file step by step, to connect the interface to a real server (API) and make the buttons work.
>
> This is the moment when everything comes together! The code we write here dictates the behavior that you, as a future Automation QA, will test with Playwright.

🟤 Step 1: DOM Selectors and the Rendering Function (Reusability)

Live Scenario: The first thing JS needs to do is get "acquainted" with the HTML elements. Then, we will create a function (a recipe) that knows how to draw an HTML row (<li>) for a task. We will write it once and reuse it both for adding a new task and for the initial data load!

🤔 ❓ How to actually test it:

- In VS Code, in the same folder, create a new file named **app.js**.

- Add the code below and save.
- Refresh the page in the browser. Nothing visual will change yet, but if you open the Console (F12) and see no red errors, it means the JS correctly "hooked" the HTML elements!

Code to write in app.js:

```
// 1. SELECTORS (Grabbing the HTML elements)
const form = document.querySelector('#add-task-form');
const inputNode = document.querySelector('#task-name');
const submitBtn = document.querySelector('#submit-task-btn');
const errorMsg = document.querySelector('#error-msg');
const taskList = document.querySelector('#task-list');
const spinner = document.querySelector('#loading-spinner');

// Our public API for testing
const API_URL = 'https://jsonplaceholder.typicode.com/todos';

// 2. REUSABLE FUNCTION: Drawing an HTML row on the page
const drawTaskInDOM = (id, title) => {
    // Create an <li> element in memory
    const li = document.createElement('li');
    li.className = 'task-item';

    // Inject the internal HTML (including testing attributes for
Playwright!)
    li.innerHTML = `
        <span class="task-title">title</span><buttonclass="btn-delete"data-
testid="delete-btn-{id}">Delete</button>
    `;

    // Physically append it to the <ul> list on the screen
    taskList.appendChild(li);
};
```

🟢 Step 2: Loading initial data on startup (GET)

Live Scenario: When you open the application, it shouldn't be empty. It must automatically fetch the latest tasks saved in the database and, very importantly, hide the orange "Loading..." banner once the data has arrived.

🤔 ❓ **How to actually test it:**

- Continue adding the code below into app.js.
- Save and refresh the page in the browser.
- **Watch the screen closely:** You will see the orange banner ("Loading data from server...") for a fraction of a second. Then, it disappears and in its place, 5 tasks fetched *live* from the internet will appear!

Code to continue in app.js:

```
// 3. GET: Fetching initial data from the Server
const loadServerData = async () => {
    try {
        // Request the first 5 tasks from the API (Waiting for the
"Waiter")
        const response = await fetch(`${API_URL}?_limit=5`);

        if (!response.ok) throw new Error("Server crashed!");

        // Unpack the JSON data
        const data = await response.json();

        // Once we have the data, we hide the spinner
        spinner.style.display = 'none';

        // Loop through the list and draw each task
        data.forEach(task => {
            drawTaskInDOM(task.id, task.title);
        });

    } catch (error) {
        spinner.textContent = "Error fetching data!";
        spinner.style.backgroundColor = "#f8d7da";
        spinner.style.color = "#721c24";
        console.error(error);
    }
```

```
};

// Start the function as soon as the script executes
loadServerData();
```

🟤 Step 3: Saving a New Task (POST & Latency)

Live Scenario: When the user fills out the form and clicks "Save", we must intercept the submission, send the payload to the server via the POST method, show a visual indicator on the button ("Saving..."), and only at the very end draw the new task in the list.

🤔 ❓ How to actually test it:

- Add this code to app.js.
- Refresh the browser.
- Because the "Save" button is disabled by default (from HTML), click on the word disabled in your index.html file and temporarily delete it just so we can test this step.
- Type something in the box and click "Save".
- Notice how the button changes its text, and after a short network latency, your task appears at the bottom of the list, and the input box empties itself!

Code to continue in app.js:

```
// 4. POST: Adding a new task over the network
form.addEventListener('submit', async (e) => {
    e.preventDefault(); // Very important: Stops the page refresh!

    const taskTitle = inputNode.value.trim();

    // UX Feedback: Block the button and change its text while waiting for
the network
    submitBtn.textContent = "Saving...";
    submitBtn.disabled = true;

    try {
        const response = await fetch(API_URL, {
            method: 'POST',
```

```
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                title: taskTitle,
                completed: false,
                userId: 1
            })
        });

        if (response.status === 201) {
            const newTask = await response.json();

            // The server gave us back the newly generated ID (e.g., 201).
Let's draw the task!
            drawTaskInDOM(newTask.id, newTask.title);

            // Clear the form
            inputNode.value = '';
        }
    } catch (error) {
        alert("Error saving over the network!");
    } finally {
        // Restore the button to its initial state, regardless of success
or error
        submitBtn.textContent = "Save";
        submitBtn.disabled = true; // Leave it disabled because the input
is now empty
    }
});
```

🟤 Step 4: Form Validation (the 'input' event)

Live Scenario: We don't want users to be able to submit empty tasks. We must "listen" to every single letter they type. If they delete everything, we block the save button and display the red error message.

🤔 ❓ How to actually test it:

- Add this final block of code to app.js (and if you deleted disabled from index.html in the previous step, put it back!).

- Save and refresh.
- Click on the input, type the word "Test". The "Save" button activates immediately (turns blue).
- Use Backspace and delete the word "Test". Instantly, the button turns grey again (disabled), and the red error message appears below it.
- This is a 100% mature Front-end feature that you, as a QA, will test automatically!

Final code to add in app.js:

```
// 5. EVENTS: "Live" validation on every keystroke
inputNode.addEventListener('input', () => {
    // Check if there is at least one letter written
    if (inputNode.value.trim().length > 0) {
        submitBtn.disabled = false;
        errorMsg.style.display = 'none';
    } else {
        // If it's empty, block the button and show the error
        submitBtn.disabled = true;
        errorMsg.style.display = 'block';
    }
});
```

🎉 **MAJOR CONGRATULATIONS!** You have finished building the E2E Task Tracker application! You now have a real, stable platform, featuring network latency, validations, and an impeccable data-testid structure. Your application is ready. In Module 4, you and the Playwright robot will launch the first automated attacks against it!

Chapter 4: Network Deletion (DELETE) and the Final QA Checklist

Our application looks and works great on the adding and reading side (GET & POST). But, for a complete CRUD scenario, we are missing the last and most destructive action: **Deletion (DELETE)**.

Furthermore, after we finish the code, we will completely switch our "Developer hat" for the "QA Engineer hat" and run a Manual Checklist before declaring the application ready for Playwright automation.

🟤 Step 1: Real Deletion (HTTP DELETE Method & Event Delegation)

Live Scenario: The "Delete" buttons are created dynamically by JavaScript (they don't exist in the initial HTML; they are drawn only after the data arrives from the server). If we try to put a classic addEventListener on them at the beginning, it will throw an error.

The solution is **Event Delegation**: we put the listener on the entire <ul> list and check if the click came from a button with the btn-delete class. Then, we will extract the task ID and send an HTTP DELETE request to the server.

🤓 ❓ How to actually test it:

- Add this final block of code to the end of your app.js file.
- Save and refresh the browser.
- Click the red "Delete" button of any task. Notice how the button gets disabled, its text changes to "Deleting...", and only after the server responds successfully does the entire row disappear from the screen.

Final code to add in app.js:

```
// 6. DELETE: Deleting a task over the network and from the DOM
taskList.addEventListener('click', async (e) => {
    // We check if the clicked element has the 'btn-delete' class
    if (e.target.classList.contains('btn-delete')) {

        const deleteBtn = e.target;
        const randTask = deleteBtn.closest('.task-item');

        // We extract the task ID from the testing attribute (e.g.,
"delete-btn-1" -> "1")
        const testId = deleteBtn.getAttribute('data-testid');
        const taskId = testId.split('-')[2];

        // UX (Visual Feedback): Block the button and show it's processing
        deleteBtn.textContent = 'Deleting...';
        deleteBtn.disabled = true;
        randTask.style.opacity = '0.6'; // Make the row look like a "ghost"

        try {
            // Send the delete request to the Server
            const response = await fetch(`
APIURL/
{taskId}`, {
                method: 'DELETE'
            });

            if (response.ok) {
                // If the server confirmed the deletion (HTTP 200/204),
physically destroy the DOM element
                randTask.remove();
            } else {
                throw new Error("Server refused the deletion!");
```

```
            }
        } catch (error) {
            // If the network drops, restore the element to its initial
state!
            alert("Network error on deletion!");
            deleteBtn.textContent = 'Delete';
            deleteBtn.disabled = false;
            randTask.style.opacity = '1';
        }
    }
});
```

🟤 Step 2: Manual QA Checklist (SUT Sign-off)

Now that you have completely built the application, you are the QA! Before writing automated code in Playwright (which we will do in Module 4), you must ensure that your SUT (Software Under Test) is highly testable and reacts correctly to human scenarios.

Manually execute these 4 fundamental tests in your browser:

☑ **Test 1: Initial Load (GET Intercept State)**

- *Action:* Open/Refresh the page.
- *Expected Result:* The orange "Loading data..." message appears. It subsequently disappears, and 5 task rows fetched live from the API show up. *(In automation, we will learn to test element states here with await expect(locator).toBeVisible())*.

☑ **Test 2: Field Validation (Boundary & State Testing)**

- *Action:* Click the text field, type a word, then delete it using Backspace until the field is completely empty.
- *Expected Result:* As soon as the field is empty, the "Save" button turns grey (disabled state), and the red error message appears below it. *(This is pure UI validation)*.

☑ **Test 3: Addition (POST & Active Waiting)**

- *Action:* Type "Playwright E2E Testing" and click Save.
- *Expected Result:* The button changes to "Saving...". After the network latency, the new row appears at the end of the list, and the text field empties automatically. *(In Playwright, we will write an assertion that waits for the list length to increase by +1 element)*.

☑ **Test 4: Deletion with Latency Handling (DELETE)**

- *Action:* Click the red "Delete" button next to a task.
- *Expected Result:* The element visually fades, the button says "Deleting...", and only after the HTTP promise is resolved does the element completely disappear. *(This is the hardest test in automation: verifying the disappearance of a DOM element using not.toBeAttached())*.

🎉 **END OF MODULE 2!**

You now know exactly how the web works under the hood. Look closely at the data-testid attributes you added in HTML and JS. They are the reason you will love this application in the upcoming modules. Playwright will never have to guess where to click; we just drew the perfect architectural map for it!

Your application is officially ready! Next up is Module 3, where we will make the ultimate upgrade: introducing **TypeScript**.

Chapter 5 (Bonus): "Chaos" Mode - The enemy of automated tests (Flaky Pop-up)

> 🧠 Our application works perfectly. But in the real world (in production), applications aren't always predictable. Cookie banners, Black Friday promos, or "Subscribe to our newsletter" windows appear, loaded via external scripts with a random delay.
> If you write an automated test that clicks "Save", and at that exact millisecond a pop-up appears over your button, the test will fail with the famous error: Element is intercepted by another element.

> Let's intentionally program this "nightmare" into our application to train our QA Automation muscles in the upcoming modules!

🟤 Step 1: Pop-up Styling (Z-Index and Overlays)

Live Scenario: The pop-up window must float above the entire application and have a semi-transparent background that temporarily blocks interaction with the rest of the page (an "Overlay").

🤔 ❓ How to actually test it:

- Open your style.css file and add the code below right at the end.
- Save the file. Visually nothing will happen yet, because the HTML element does not exist. We are just preparing the ground for JavaScript.

Final code to add in style.css:

```
/* 7. Chaos Mode: Flaky Promo Pop-up */
.overlay-modal {
    position: fixed;
    top: 0; left: 0; right: 0; bottom: 0;
    background-color: rgba(0, 0, 0, 0.7); /* Dark transparent background */
    z-index: 9999; /* Making sure it's above absolutely all elements */
    display: flex;
    justify-content: center;
    align-items: center;
    backdrop-filter: blur(3px);
}

.promo-box {
    background: white;
    padding: 30px;
    border-radius: 10px;
    text-align: center;
    box-shadow: 0 10px 25px rgba(0,0,0,0.5);
    max-width: 400px;
    animation: popIn 0.3s ease-out forwards; /* A small visual animation */
}
```

```
.promo-box h3 { color: #e74c3c; margin-bottom: 15px; font-size: 22px; }
.promo-box p { margin-bottom: 20px; font-size: 15px; }

.btn-close-promo {
    background-color: #2c3e50;
    color: white;
    width: 100%;
}
.btn-close-promo:hover:not(:disabled) { background-color: #1a252f; }

@keyframes popIn {
    0% { transform: scale(0.5); opacity: 0; }
    100% { transform: scale(1); opacity: 1; }
}
```

🥒 Step 2: Random Asynchronous JavaScript (The Injection)

Live Scenario: We will use the setTimeout method combined with the Math.random() function. We will tell the application: "After a random time between 2 and 6 seconds from the page opening, suddenly generate the Promo window. Also add a close button that destroys the window."

🤔 ❓ How to actually test it:

- Open the app.js file and paste this block of code exactly at the end of the file.
- Save and **Refresh** the browser.
- Try to add a new task. Type quickly, press "Save". Refresh and try again.
- At some point (between 2 and 6 seconds), exactly when you least expect it, the screen will darken and the Promotional message will appear, blocking your actions. Click "No, thanks" to close it and resume your work.

Final code to add in app.js:

```
// 7. BONUS: Chaos Mode - Random Pop-up that intercepts UI tests!
const initChaosMode = () => {
    // Calculate a random waiting time between 2000ms and 6000ms
    const randomDelay = Math.floor(Math.random() * 4000) + 2000;

    setTimeout(() => {
        // Create the container element (The Overlay)
        const modal = document.createElement('div');
        modal.className = 'overlay-modal';
```

```
        modal.setAttribute('data-testid', 'promo-overlay');

        // Inject the promo box and the close button
        modal.innerHTML = `
            <div class="promo-box">
                <h3>Special Offer! 🚀</h3>
                <p>Upgrade to the Premium version to add unlimited
tasks!</p>
                <button class="btn-close-promo" data-testid="close-promo-
btn">No, thanks</button>
            </div>
        `;

        // Attach the closing event
        const closeBtn = modal.querySelector('.btn-close-promo');
        closeBtn.addEventListener('click', () => {
            modal.remove(); // Destroy the modal when the user clicks
        });

        // Physically "throw" the modal into the page (at the end of the
<body> tag)
        document.body.appendChild(modal);

        console.warn(`[Chaos Mode] Promo modal appeared after
${randomDelay}ms!`);
    }, randomDelay);
};

// Start Chaos mode!
initChaosMode();
```

🟤 QA Conclusion (SUT Sign-off)

📈 What you just did is not a simple game. It is a classic **Edge Case** in Software Engineering. When we write the automated tests in Playwright, your robot will have to be smart enough to make a decision:

- *"I want to click Save. Is there a modal on the screen blocking me? If so, I close it first, and only then click Save."*

Playwright has a special function for this (called page.addLocatorHandler()), which acts as an asynchronous "watchdog" that hunts these unstable windows during the test run. We just built

> the perfect training ground for it!